Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа №3

на тему:

«Критерии выбора оптимальных стратегий в бескоалиционных играх нескольких игроков»

Вариант 1

Студент: ИУ8-105 (Группа) В. В. Гоза (И. О. Фамилия)

Преподаватель: Н. С. Коннова (И.О. Фамилия)

Москва, 2026

Цель работы

Изучить критерии выбора стратегий в неантагонистической бескоалиционной игре двух игроков на основе равновесия Нэша и оптимальности по Парето. Проверить данные критерии на примере игр «Перекресток», «Семейный спор», «Дилемма заключенного». Исследовать свойства оптимальных решений неантагонистических бескоалиционных игр на примере биматричных (2 x 2)-игр.

Постановка задачи

1. Сгенерировать случайную биматричную игру (10 x 10). Найти ситуации, равновесные по Нэшу и оптимальные по Парето, а также пересечение множеств этих ситуаций. Выполнить проверку реализованных алгоритмов на примере трех известных игр: «Семейный спор», «Перекресток», «Дилемма заключенного».
2. Для заданной биматричной (2 x 2)-игры Г(A, B), пользуясь теоремами о свойствах оптимальных решений, найти ситуации, равновесные по Нэшу, для исходной игры и для ее смешанного расширения.

1	$\begin{bmatrix} (5, 0) & (8, 4) \\ (7, 6) & (6, 3) \end{bmatrix}$

Рисунок 1 – Условие задачи для варианта 1

Ход работы

На рисунке 1 изображена случайная биматричная игра (10x10). В ней зелёным цветом обозначены стратегии равновесные по Нэшу, жёлтым цветом оптимальные по Парето, а розовым цветом – их пересечение. Все такие ситуации изображены на рисунке 2.

Биматричная игра 10x10:

	β1	β2	β3	β4	β5	β6	β7	β8	β9	β10
α1	(12, -7)	(-18, 15)	(5, -11)	(22, 8)	(3, 39)	(14, -20)	(31, 12)	(-25, 5)	(7, -14)	(38, 0)
α2	(42, 18)	(27, -9)	(-16, 6)	(11, -13)	(24, 10)	(-21, 4)	(8, -5)	(16, 37)	(-12, -2)	(29, 7)
α3	(-14, 9)	(35, 4)	(18, -8)	(-7, 11)	(26, -1)	(9, 16)	(47, 40)	(13, -6)	(-19, 5)	(21, 2)
α4	(25, 38)	(6, 12)	(44, 3)	(17, -15)	(4, 20)	(30, -7)	(-8, 9)	(12, -10)	(44, 6)	(-5, 1)
α5	(10, 5)	(-22, -3)	(14, 18)	(33, 36)	(-6, -12)	(41, 13)	(5, -9)	(27, 4)	(-15, 16)	(8, -1)
α6	(-9, 14)	(19, 6)	(2, -5)	(15, 17)	(34, 1)	(-4, -11)	(11, 8)	(23, -6)	(39, 48)	(-17, 3)
α7	(28, 2)	(-13, 10)	(9, -14)	(45, 6)	(17, -4)	(32, 34)	(-5, 7)	(21, -8)	(14, 11)	(0, -9)
α8	(7, 13)	(43, 35)	(-18, 4)	(26, -2)	(12, 21)	(-7, 5)	(16, -10)	(3, 8)	(30, 1)	(-11, 17)
α9	(18, -6)	(5, 9)	(24, 12)	(-20, 3)	(46, -5)	(8, 14)	(2, 6)	(35, -7)	(-4, 18)	(15, 33)
α10	(-16, 11)	(29, -1)	(6, 31)	(20, -9)	(3, 18)	(25, 2)	(-14, 13)	(40, -4)	(22, 15)	(-8, 5)

Рисунок 1 – Случайная биматричная игра (10x10).

```
Равновесия Нэша в чистых стратегиях:
  (α3, β7) -> (47, 40)
  (α8, β2) -> (43, 35)

Парето-оптимальные ситуации:
  (α3, β7) -> (47, 40)
  (α6, β9) -> (39, 48)

Пересечение множеств:
  (α3, β7) -> (47, 40)
```

Рисунок 2 – Выбранные стратегии

На рисунке 3, 4, 5 изображена работа реализованных алгоритмов примере игр «Семейный спор», «Перекресток» и «Дилемма заключенного», соответственно. Реализованные алгоритмы работают корректно, это подтверждается результатами игр.

```
Проверка алгоритма: игра «Семейный спор»

   |      β1      |      β2
--------------------------------
α1 |    (4, 1)    |    (0, 0)
α2 |    (0, 0)    |    (1, 4)

Равновесия Нэша в чистых стратегиях:
  (α1, β1) -> (4, 1)
  (α2, β2) -> (1, 4)

Парето-оптимальные ситуации:
  (α1, β1) -> (4, 1)
  (α2, β2) -> (1, 4)

Пересечение множеств:
  (α1, β1) -> (4, 1)
  (α2, β2) -> (1, 4)
```

Рисунок 3 – Игра «Семейный спор»

```
   |      β1      |      β2
--------------------------------
α1 |    (1, 1)    |  (1.00, 2)
α2 |  (2, 1.00)   |    (0, 0)

Равновесия Нэша в чистых стратегиях:
  (α1, β2) -> (1.00, 2)
  (α2, β1) -> (2, 1.00)

Парето-оптимальные ситуации:
  (α1, β1) -> (1, 1)
  (α1, β2) -> (1.00, 2)
  (α2, β1) -> (2, 1.00)

Пересечение множеств:
  (α1, β2) -> (1.00, 2)
  (α2, β1) -> (2, 1.00)
```

Рисунок 4 – Игра «Перекресток»

```
   |      β1      |      β2
--------------------------------
α1 |   (-5, -5)   |   (0, -10)
α2 |   (-10, 0)   |   (-1, -1)

Равновесия Нэша в чистых стратегиях:
  (α1, β1) -> (-5, -5)

Парето-оптимальные ситуации:
  (α1, β2) -> (0, -10)
  (α2, β1) -> (-10, 0)
  (α2, β2) -> (-1, -1)

Пересечение множеств:
  нет
```

Рисунок 5 – Игра «Дилемма заключенного»

Заданная биматричная (2x2)-игра, имеет две равновесные по Нэшу ситуации (см. рисунок 6), тогда в смешанном дополнении игры существует еще одна вполне смешанная ситуация равновесия (x^*, y^*) (см. рисунок 7).

```
Вариант 1 из задания:
   |      β1      |      β2
--------------------------------
α1 |    (5, 0)    |    (8, 4)
α2 |    (7, 6)    |    (6, 3)

Равновесия Нэша в чистых стратегиях:
  (α1, β2) -> (8, 4)
  (α2, β1) -> (7, 6)

Парето-оптимальные ситуации:
  (α1, β2) -> (8, 4)
  (α2, β1) -> (7, 6)

Пересечение множеств:
  (α1, β2) -> (8, 4)
  (α2, β1) -> (7, 6)
```

Рисунок 6 – Биматричная игра (2x2)

```
Смешанное расширение:
  x* = (3/7, 4/7) ≈ (0.428571, 0.571429)
  y* = (1/2, 1/2) ≈ (0.500000, 0.500000)
  v1 = 13/2 ≈ 6.500000
  v2 = 24/7 ≈ 3.428571
```

Рисунок 7 – Вполне смешанное равновесие Нэша

Вывод

В ходе выполнения лабораторной работы были изучены критерии выбора оптимальных стратегий в неантагонистических бескоалиционных играх нескольких игроков. Для случайно сгенерированной биматричной игры 10×10 были найдены ситуации, равновесные по Нэшу, оптимальные по Парето, а также пересечение этих множеств.

Также была выполнена проверка реализованных алгоритмов на известных играх: «Семейный спор», «Перекресток» и «Дилемма заключенного». Полученные результаты совпали с теоретически ожидаемыми, что подтверждает корректность реализации алгоритмов поиска равновесий по Нэшу и Парето-оптимальных ситуаций.

Кроме того, для заданной биматричной игры 2×2 были найдены равновесные по Нэшу ситуации в чистых стратегиях и исследовано смешанное расширение игры. Было показано, что помимо равновесий в чистых стратегиях может существовать вполне смешанное равновесие Нэша. Код программы, с помощью которой проводились вычисления представлен в Приложении А.

Приложение А

Листинг (также приведён по ссылке https://github.com/NeAlien/Game_theory_10_sem):

```
from __future__ import annotations

from fractions import Fraction
from typing import List, Tuple, Optional, Set

Payoff = Tuple[float, float]
Game = List[List[Payoff]]


RESET = "\033[0m"
BOLD = "\033[1m"

COLOR_NASH = "\033[92m"
COLOR_PARETO = "\033[93m"
COLOR_BOTH = "\033[95m"


USE_COLORS = True


def colorize(text: str, color: str) -> str:
    if not USE_COLORS:
        return text
    return f"{color}{text}{RESET}"


def to_fraction_str(x: Fraction | float) -> str:
    if isinstance(x, Fraction):
        if x.denominator == 1:
            return str(x.numerator)
        return f"{x.numerator}/{x.denominator}"
    return f"{x:.6f}"


def payoff_str(p: Payoff) -> str:
    a, b = p

    if float(a).is_integer():
        a_str = str(int(a))
    else:
        a_str = f"{a:.2f}"
```

```
    if float(b).is_integer():
        b_str = str(int(b))
    else:
        b_str = f"{b:.2f}"

    return f"({a_str}, {b_str})"


def extract_A(game: Game) -> List[List[float]]:
    return [[cell[0] for cell in row] for row in game]


def extract_B(game: Game) -> List[List[float]]:
    return [[cell[1] for cell in row] for row in game]


def nash_equilibria_pure(game: Game) -> List[Tuple[int, int]]:
    A = extract_A(game)
    B = extract_B(game)
    m = len(A)
    n = len(A[0])

    best_rows_for_col = []
    for j in range(n):
        col_values = [A[i][j] for i in range(m)]
        best_value = max(col_values)
        best_rows = {i for i in range(m) if A[i][j] == best_value}
        best_rows_for_col.append(best_rows)

    best_cols_for_row = []
    for i in range(m):
        row_values = [B[i][j] for j in range(n)]
        best_value = max(row_values)
        best_cols = {j for j in range(n) if B[i][j] == best_value}
        best_cols_for_row.append(best_cols)

    result = []
    for i in range(m):
        for j in range(n):
            if i in best_rows_for_col[j] and j in best_cols_for_row[i]:
                result.append((i, j))

    return result
```

```
def pareto_optimal_profiles(game: Game) -> List[Tuple[int, int]]:
    m = len(game)
    n = len(game[0])

    result = []
    for i in range(m):
        for j in range(n):
            current = game[i][j]
            dominated = False

            for r in range(m):
                for c in range(n):
                    other = game[r][c]
                    if (
                        other[0] >= current[0]
                        and other[1] >= current[1]
                        and (other[0] > current[0] or other[1] > current[1])
                    ):
                        dominated = True
                        break
                if dominated:
                    break

            if not dominated:
                result.append((i, j))

    return result


def intersection_of_profiles(
    first: List[Tuple[int, int]],
    second: List[Tuple[int, int]],
) -> List[Tuple[int, int]]:
    return sorted(list(set(first) & set(second)))


def print_legend() -> None:
    print("Обозначения:")
    print("  " + colorize("■", COLOR_NASH) + " — равновесие по Нэшу")
    print("  " + colorize("■", COLOR_PARETO) + " — Парето-оптимальная ситуация")
```

```
    print("  " + colorize("■", COLOR_BOTH) + " — одновременно Нэш и Парето")
    print()


def print_game(
    game: Game,
    title: str = "Биматричная игра",
    nash_profiles: Optional[List[Tuple[int, int]]] = None,
    pareto_profiles: Optional[List[Tuple[int, int]]] = None,
) -> None:
    print(f"\n{title}")


    m = len(game)
    n = len(game[0])

    nash_set: Set[Tuple[int, int]] = set(nash_profiles or [])
    pareto_set: Set[Tuple[int, int]] = set(pareto_profiles or [])
    both_set: Set[Tuple[int, int]] = nash_set & pareto_set

    row_labels = [f"α{i + 1}" for i in range(m)]
    col_labels = [f"β{j + 1}" for j in range(n)]

    cell_text = [[payoff_str(game[i][j]) for j in range(n)] for i in range(m)]

    cell_width = max(
        max(len(x) for row in cell_text for x in row),
        max(len(x) for x in col_labels),
        12
    )
    row_label_width = max(len(x) for x in row_labels)

    matrix_width = n * cell_width + (n - 1) * 3


    header = " " * row_label_width + " | " + " | ".join(
        label.center(cell_width) for label in col_labels
    )
    print(header)
    print("-" * len(header))

    for i in range(m):
        cells = []
```

```
        for j in range(n):
            raw = cell_text[i][j].center(cell_width)

            if (i, j) in both_set:
                raw = colorize(raw, COLOR_BOTH)
            elif (i, j) in nash_set:
                raw = colorize(raw, COLOR_NASH)
            elif (i, j) in pareto_set:
                raw = colorize(raw, COLOR_PARETO)

            cells.append(raw)

        row = row_labels[i].rjust(row_label_width) + " | " + " | ".join(cells)
        print(row)



def print_profiles(game: Game, profiles: List[Tuple[int, int]], title: str) -> None:
    print(f"\n{title}:")
    if not profiles:
        print("  нет")
        return

    for i, j in profiles:
        print(f"  (α{i + 1}, β{j + 1}) -> {payoff_str(game[i][j])}")


def analyze_game_by_nash_pareto(game: Game, title: str) -> None:
    pure_nash = nash_equilibria_pure(game)
    pareto = pareto_optimal_profiles(game)
    both = intersection_of_profiles(pure_nash, pareto)

    print_game(
        game,
        title,
        nash_profiles=pure_nash,
        pareto_profiles=pareto,
    )

    print_profiles(game, pure_nash, "Равновесия Нэша в чистых стратегиях")
    print_profiles(game, pareto, "Парето-оптимальные ситуации")
    print_profiles(game, both, "Пересечение множеств")
```

```
def strictly_dominant_rows(game: Game) -> List[int]:
    A = extract_A(game)
    m = len(A)
    n = len(A[0])

    dominant = []
    for i in range(m):
        ok = True
        for k in range(m):
            if i == k:
                continue
            if not all(A[i][j] > A[k][j] for j in range(n)):
                ok = False
                break
        if ok:
            dominant.append(i)
    return dominant


def strictly_dominant_cols(game: Game) -> List[int]:
    B = extract_B(game)
    m = len(B)
    n = len(B[0])

    dominant = []
    for j in range(n):
        ok = True
        for k in range(n):
            if j == k:
                continue
            if not all(B[i][j] > B[i][k] for i in range(m)):
                ok = False
                break
        if ok:
            dominant.append(j)
    return dominant


def mixed_equilibrium_2x2(game: Game) -> Optional[dict]:
    if len(game) != 2 or len(game[0]) != 2:
        raise ValueError("Функция mixed_equilibrium_2x2 работает только для игр 2×2.")

    a11, b11 = game[0][0]
    a12, b12 = game[0][1]
```

```
    a21, b21 = game[1][0]
    a22, b22 = game[1][1]

    den_q = Fraction(a11) - Fraction(a12) - Fraction(a21) + Fraction(a22)
    den_p = Fraction(b11) - Fraction(b12) - Fraction(b21) + Fraction(b22)

    if den_q == 0 or den_p == 0:
        return None

    q = (Fraction(a22) - Fraction(a12)) / den_q
    p = (Fraction(b22) - Fraction(b21)) / den_p

    if not (Fraction(0) <= p <= Fraction(1) and Fraction(0) <= q <= Fraction(1)):
        return None

    x = (p, Fraction(1) - p)
    y = (q, Fraction(1) - q)

    v1 = q * Fraction(a11) + (Fraction(1) - q) * Fraction(a12)
    v2 = p * Fraction(b11) + (Fraction(1) - p) * Fraction(b21)

    return {
        "p": p,
        "q": q,
        "x": x,
        "y": y,
        "v1": v1,
        "v2": v2,
    }


def analyze_variant_2x2(game: Game, title: str) -> None:
    analyze_game_by_nash_pareto(game, title)

    dom_rows = strictly_dominant_rows(game)
    dom_cols = strictly_dominant_cols(game)

    print("\nПроверка строго доминирующих стратегий:")
    print("  Игрок 1:", "нет" if not dom_rows else ", ".join(f"α{i + 1}" for i in dom_rows))
    print("  Игрок 2:", "нет" if not dom_cols else ", ".join(f"β{j + 1}" for j in dom_cols))

    mixed = mixed_equilibrium_2x2(game)
    print("\nСмешанное расширение:")
    if mixed is None:
```

```
        print("  Вполне смешанное равновесие не существует или не является внутренним.")
    else:
        x1, x2 = mixed["x"]
        y1, y2 = mixed["y"]
        print(
            f"  x* = ({to_fraction_str(x1)}, {to_fraction_str(x2)})"
            f" ≈ ({float(x1):.6f}, {float(x2):.6f})"
        )
        print(
            f"  y* = ({to_fraction_str(y1)}, {to_fraction_str(y2)})"
            f" ≈ ({float(y1):.6f}, {float(y2):.6f})"
        )
        print(f"  v1 = {to_fraction_str(mixed['v1'])} ≈ {float(mixed['v1']):.6f}")
        print(f"  v2 = {to_fraction_str(mixed['v2'])} ≈ {float(mixed['v2']):.6f}")


def example_matrix_10x10() -> Game:
    return [
        [(12, -7),  (-18, 15), (5, -11),  (22, 8),   (3, 39),   (14, -20), (31, 12),  (-25, 5),  (7, -14),  (38,
0)],
        [(42, 18),  (27, -9),  (-16, 6),  (11, -13), (24, 10),  (-21, 4),  (8, -5),   (16, 37),  (-12, -2), (29,
7)],
        [(-14, 9),  (35, 4),   (18, -8),  (-7, 11),  (26, -1),  (9, 16),   (47, 40),  (13, -6),  (-19, 5),  (21, 2)],
        [(25, 38),  (6, 12),   (44, 3),   (17, -15), (4, 20),   (30, -7),  (-8, 9),   (12, -10), (44, 6),   (-5, 1)],
        [(10, 5),   (-22, -3), (14, 18),  (33, 36),  (-6, -12), (41, 13),  (5, -9),   (27, 4),   (-15, 16), (8, -
1)],
        [(-9, 14),  (19, 6),   (2, -5),   (15, 17),  (34, 1),   (-4, -11), (11, 8),   (23, -6),  (39, 48),  (-17,
3)],
        [(28, 2),   (-13, 10), (9, -14),  (45, 6),   (17, -4),  (32, 34),  (-5, 7),   (21, -8),  (14, 11),  (0, -9)],
        [(7, 13),   (43, 35),  (-18, 4),  (26, -2),  (12, 21),  (-7, 5),   (16, -10), (3, 8),    (30, 1),   (-11,
17)],
        [(18, -6),  (5, 9),    (24, 12),  (-20, 3),  (46, -5),  (8, 14),   (2, 6),    (35, -7),  (-4, 18),  (15, 33)],
        [(-16, 11), (29, -1),  (6, 31),   (20, -9),  (3, 18),   (25, 2),   (-14, 13), (40, -4),  (22, 15),  (-8,
5)],
    ]


def family_dispute_game() -> Game:
    return [
        [(4, 1), (0, 0)],
        [(0, 0), (1, 4)],
    ]
```

```
def intersection_game() -> Game:
    eps = 0.001
    return [
        [(1, 1), (1 - eps, 2)],
        [(2, 1 - eps), (0, 0)],
    ]


def prisoners_dilemma_game() -> Game:
    return [
        [(-5, -5), (0, -10)],
        [(-10, 0), (-1, -1)],
    ]


def verification_games() -> List[Tuple[str, Game]]:
    return [
        ("Проверка алгоритма: игра «Семейный спор»\n", family_dispute_game()),
        ("Проверка алгоритма: игра «Перекрёсток со смещением»\n", intersection_game()),
        ("Проверка алгоритма: игра «Дилемма заключённого»\n", prisoners_dilemma_game()),
    ]


def var_1() -> Game:
    return [
        [(5, 0), (8, 4)],
        [(7, 6), (6, 3)],
    ]


def main() -> None:
    main_game = example_matrix_10x10()
    analyze_game_by_nash_pareto(main_game, "Биматричная игра 10×10:\n")

    print("\nПРОВЕРКА ТЕХ ЖЕ АЛГОРИТМОВ НА ИЗВЕСТНЫХ ИГРАХ")

    for title, game in verification_games():
        analyze_game_by_nash_pareto(game, title)

    game_v1 = var_1()
    analyze_variant_2x2(game_v1, "Вариант 1 из задания:")


if __name__ == "__main__":
```

main()